From operational CDC to trusted analytics

A measured Spark Structured Streaming platform for ACME Delivery Services

Fabricio Dutra

Architecture • methodology • sizing • evidence

Implemented end to end. Tested for correctness, load and recovery.

Sources: README.md | docs/assessment-deliverables.md

BUSINESS OUTCOME

Four questions drive the platform

01 Open orders

By delivery date and status
Support workload planning.

02 Top delivery dates

The three dates with most open orders
Identify delivery-day concentration.

03 Pending items

By product: item lines and total units
Support picking and storage priorities.

04 Top customers

The three customers with most pending orders
Prioritize operational follow-up.

Definition: PENDING, PROCESSING and REPROCESSING are open; COMPLETED is excluded.

Sources: sql/open-orders.sql | sql/pending-items.sql

METHODOLOGY

Make the assumptions explicit

Historical contract

Initial snapshot + current state accumulated from earlier CDC batches.
No pre-snapshot reconstruction.

Freshness contract

Continuous microbatches.
30s is a test reference, not a requirement supplied by Deel.

Deployment scope

Reproducible local Docker stack.
Production sizing remains a measurement-driven strategy.

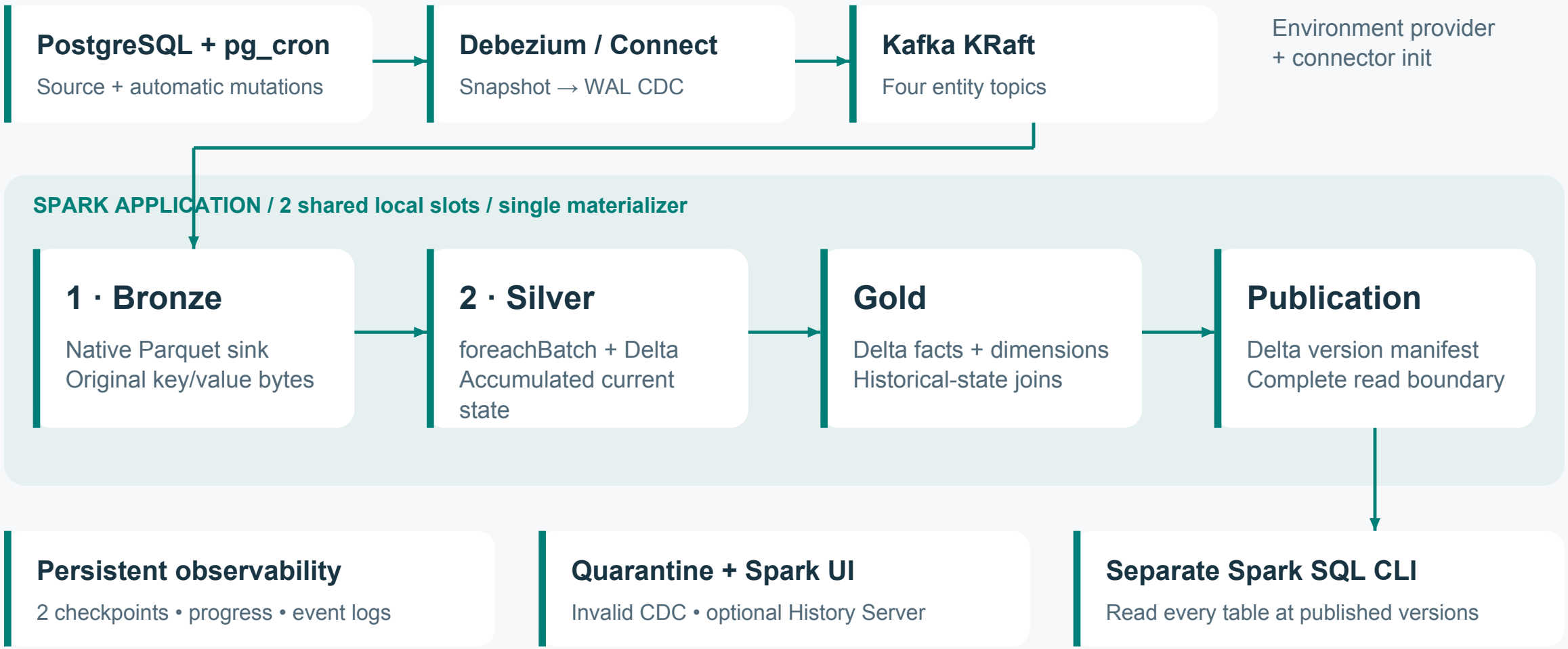
Measured separately

Correctness, bulk materialization, CDC arrival rate, publication age, recovery and query cold start.

Sources: docs/assessment-deliverables.md | docs/architecture.md

ARCHITECTURE

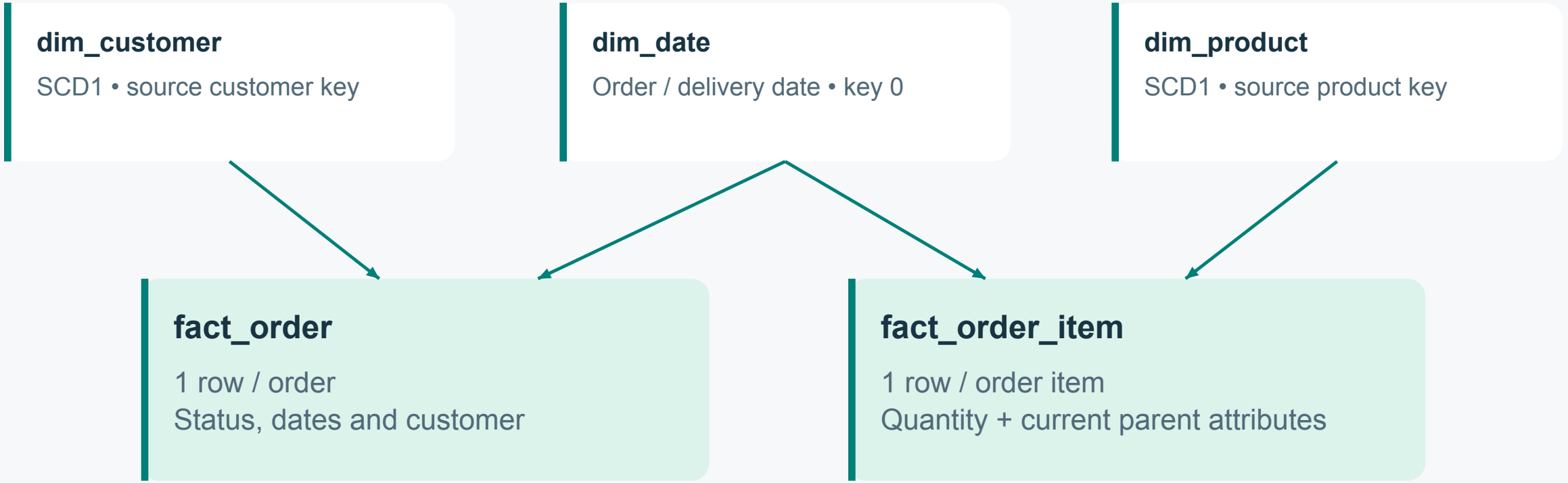
One complete source-to-query path



Sources: docs/architecture.md | docker-compose.yaml

DIMENSIONAL MODEL

Two fact grains prevent double counting



Order metrics query fact_order directly. Item metrics count lines and sum quantities.

The source field “quanity” becomes quantity; prices use explicit DECIMAL(20,4) casts.

Sources: app/gold.py | docs/architecture.md

PROCESSING

Every microbatch earns its publication

1 Preserve

Kafka → native Parquet
Checkpoint + sink metadata

2 Validate

Decode schema and business rules
Invalid CDC → quarantine + fail

3 Order + apply

Latest LSN / Kafka offset
Conditional Silver MERGE

4 Enrich history

Changed items ↔ stored orders
Changed orders ↔ stored items

5 Refresh Gold

Affected facts and SCD1 dimensions
Retain logical delete guards

6 Publish

Commit complete table versions
CLI uses versionAsOf

PySpark DataFrames + Delta MERGE for processing; Spark SQL for the four serving queries.

Sources: app/materialize.py | app/storage.py

Historical joins are part of the stream

01

Snapshot baseline

Event → historical lookup

Order O1 is PENDING

Store the order in Silver

Order exists in Gold

02

Later item event

Event → historical lookup

New item I1 references O1

Join I1 with stored order O1

Item inherits parent state

03

Later parent update

Event → historical lookup

Order O1 becomes COMPLETED

Retrieve its stored item I1

Pending item disappears

Illustrative sequence based on tested behavior. “History” means retained state, not SCD2.

Sources: app/gold.py | docs/evidence/historical-join-tests.log

Choose the simplest components that fit

Decision	Reason	Tradeoff
Parquet Bronze	Preserve original events; native append sink	Keep files, metadata and checkpoint together
Delta Silver / Gold	Ordered upserts, delete guards, versioned reads	MERGE and retained-target scans have cost
Publication manifest	Expose a completed set of table versions	Supported readers must honor the manifest
On-demand SQL CLI	Deliver the four required queries with few services	Cold start; not a high-QPS query server

Security: environment-based secrets, restricted CDC account, unprivileged Spark, loopback UIs.

TLS, replicated services and shared durable storage are production additions.

Sources: docs/architecture.md | README.md

SPARK SIZING

Size for working sets and recovery

2 slots
Shared by both streams
local[2] • two-CPU quota

2 GiB
JVM heap
3 GiB total container cap

4
Shuffle partitions
Starting choice, not proven optimum

Budget / control	Local configuration	Sizing implication
Kafka / Connect / PostgreSQL	1 GiB / 1 GiB / 512 MiB	Fit source, CDC and Spark in ~8 GB VM
Streaming intake	20k offsets / 10s; 32 files / batch	Bounds intake, not fanout or task memory
Backlog recovery	Required rate ≥ λ + B / R	Arrival λ; backlog B; recovery window R

Scale from measured CPU, GC/spill, task balance, target scans and freshness—not row count alone.

Sources: docs/performance.md | docker-compose.yaml

VALIDATION METHOD

Triangulate correctness and performance

Semantic tests

Snapshot + CDC • ordering • deletes
Historical joins • quarantine • crash / replay

Independent source oracle

Compare all four business queries against PostgreSQL before / after changes.

Separate workloads

Bulk fixture • intake bursts • stepped CDC
Hot parent • query concurrency • outage

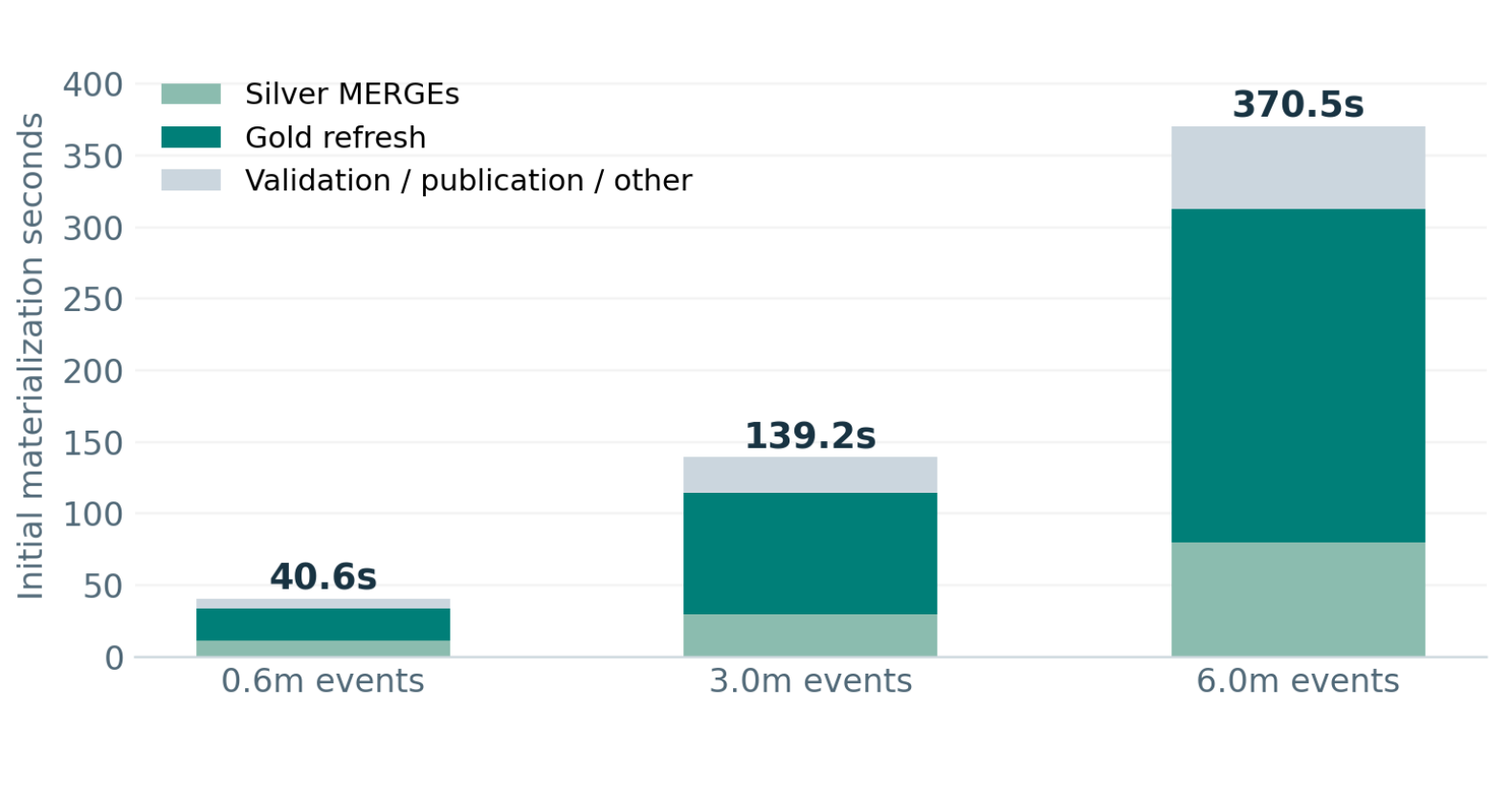
Evidence and accounting

Native task logs + stream progress
Unique Kafka coordinates + publication counts

Synthetic data; short steps; actual rates reported; no production SLA inferred.

Sources: tests/test_pipeline.py | docs/evidence/README.md

Gold dominates as retained state grows



370.5s

6,000,200 input events

Initial materialization

63%

Time in Gold refresh

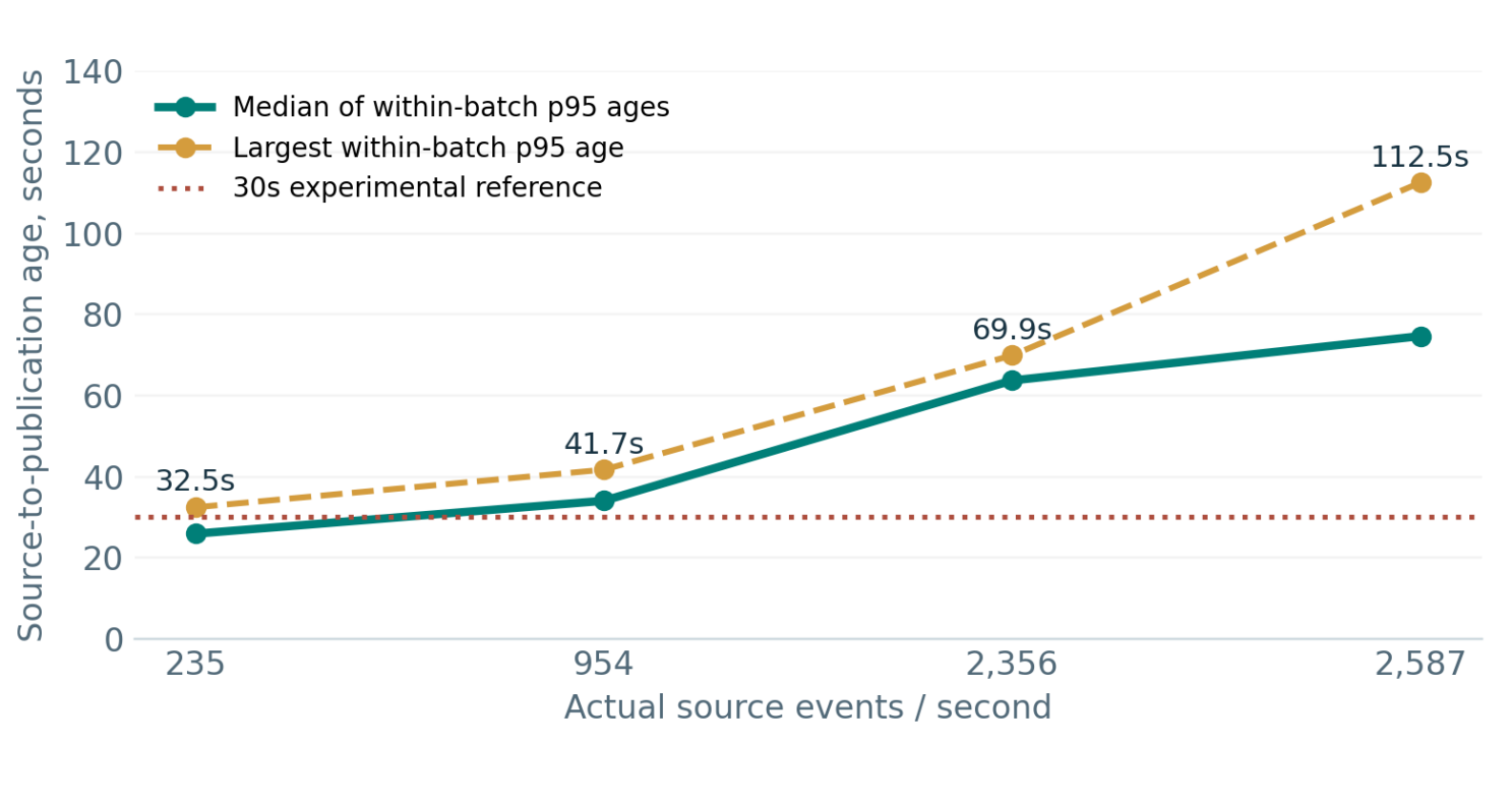
232.5s of 370.5s

846 MB disk spill
Memory pressure near cap
0 failed task attempts

Doubling ~3m → ~6m events increased initial materialization time by 2.66×.

Sources: docs/evidence/stress/materialization.json | docs/evidence/stress/scale-1m-spark-metrics.json

Freshness degrades without container failures



Reference breached

235 events/s: largest batch p95 = 32.5s
2,587 events/s: 112.5s

Catch-up observed

All four steps drained.
Highest two steps: ~66s and ~70s after load stopped.

Per-batch age summaries—not global event p95. Steps ~2 minutes; nominal 6k/s was not reached.

Sources: docs/evidence/stress/streaming-levels.json | docs/evidence/stress/streaming-batches.json

INTEGRITY / RECOVERY

Correctness survives a temporary broker outage

774,001
Kafka events = Bronze rows
Also unique Kafka coordinates

3 / 3
Metric reconciliations passed
All four business questions

53.5s
Observed recovery interval
Stop → restart → catch-up → poll

Mutations exercised

12,375 order updates
6,378 item deletes + tombstones
6,000 source events while Kafka was down

Final source / Gold agreement

124,756 orders
617,528 active items
3,167,520 units

Retained volumes / offsets / checkpoints. This is recovery evidence, not single-broker HA.

Sources: docs/evidence/stress/final-integrity.json | docs/evidence/stress/broker-outage.jsonl

NEXT EXPERIMENTS

Scale the measured constraint first

Observed constraint	Candidate action—not yet validated	Acceptance measurement
Gold / target work dominates	Benchmark layout, compaction and affected-row access	Files scanned + batch time + correctness
Heap / CPU pressure	Compare task input, memory, cores and shuffle count	Spill, GC, queued tasks and freshness
Backlog and shared slots	Tune intake with downstream capacity; isolate resources	Stable lag and catch-up within target R
Production durability / serving	Shared lake, replicated Kafka; persistent serving	Outage tests and latency under concurrency

One parent update refreshed 1m children in 14.1s: event count alone does not size the workload.

Sources: docs/performance-stress.md | docs/performance.md

Inspect, reproduce and discuss

```
python3 scripts/setup.py

docker compose up -d --build

docker compose run --rm --no-deps query metrics all
```

Wait for the first published batch, then inspect the four metrics and publication timestamp.

Working system + measurable limits + a defensible path to production sizing.

Code and all retained analysis: fabdutra/deel-data-engineering-task

Appendix: CDC semantics, failure handling, intake / serving, native metrics and evidence.

Sources: README.md | docs/evidence/README.md

Upserts and deletes preserve ordering

Layer / table	Identity	Mutation behavior	Delete behavior
Silver: 4 entities	Source primary key	Latest LSN / offset; conditional MERGE	Retain _deleted ordering guard
Gold: fact_order	order_id	Refresh changed orders	Logical deletion; CLI filters
Gold: fact_order_item	order_item_id	Changed items + stored children of changed parents	Parent/item state drives visibility
Gold: customer / product	Stable source key	SCD1 MERGE	Retain delete marker
Gold: dim_date	date_key	Merge affected date members; unknown = 0	Members retained

Topic repartitioning or source-identity changes require a controlled ordering migration.

Sources: app/cdc.py | app/storage.py

Publish a complete read boundary

Failure point	What supported readers see	Recovery action
Before / during Silver merge	Previous complete publication	Retain checkpoint; correct failure; replay
After Silver / partial Gold	Previous published Gold versions	Recompute affected Gold rows; publish last
After manifest commit	New complete version set	Checkpoint replay remains convergent
Invalid CDC contract	Previous publication; batch withheld	Inspect quarantine; fix parser / contract

No global atomic buffering of source transactions across topics. One materializer owns the tables.

Missing Kafka offsets fail visibly. Checkpoints and raw Parquet metadata must stay intact.

Sources: app/materialize.py | app/metrics.py

Intake and serving have different limits

Kafka → Bronze only

Payload	20k cap	200k cap
500 B / 250k	127.8s	20.9s
5k B / 100k	57.6s	20.4s

Fresh query containers

Concurrent processes	Group time
1	24.6s
2	37.9s
4	65.2s

What this demonstrates

The original admission cap limits these bursts; larger intake is not a Gold fix.

What remains open

Query concurrency at peak write load and an always-on serving comparison.

Sources: docs/evidence/stress/bronze-load-levels.jsonl | docs/evidence/stress/query-concurrency.jsonl

Native metrics support the sizing discussion

Fixture	Tasks / fail	CPU sum	GC sum	Disk spill	Task p95
100k orders	642 / 0	47.0s	2.9s	0.0 MB	0.497s
500k orders	689 / 0	196.3s	12.1s	137.7 MB	1.631s
1m orders	680 / 0	482.8s	51.3s	846.0 MB	6.414s

Largest offline fixture

~3 GiB cgroup peak; CPU throttling
No OOM; measurable memory pressure.

Streaming task extract

11,487 attempts / 0 failures
337.4s CPU; 22.7s task GC; no spill.

Sources: docs/evidence/stress/scale-1m-spark-metrics.json | docs/evidence/stress/online-spark-metrics.json

Captured business output is inspectable

Publication batch 39 • 2026-10-20 03:49:33.510811 UTC

Metric	Selected actual group	Result
Open orders	2026-10-20 / PENDING	33 orders
Top delivery date	2026-10-31	4,159 orders
Pending items	Product 1	556,139 lines 2,782,894 units
Top customer	Customer 1 — Awesome Comapany 2.0	112,375 orders

Selected rows only. Full outputs and independent source comparisons are retained as JSON.

Sources: docs/evidence/stress/query-output.json | docs/evidence/stress/reconciliation.jsonl

Limits are part of the analytical result

Contract limits

Current state, not SCD2
Per-table transactions + manifest reads
Single materializer; local durable volumes

Measurement limits

One run / scenario; ~2-minute steps
Actual peak input ~2.59k/s, not 6k/s
Batch p95 summaries ≠ global event p95

Replay observed

Initial 185-row snapshot emitted twice.
Bronze retained both coordinate sets.
Final checks used actual offsets; business-state reconciliation passed.

Production questions still open

Long-duration SLA and workload mix
Distributed executor scaling curve
Broker / CDC / source saturation
Remote storage and peak query writes

Sources: docs/performance-stress.md | docs/assessment-deliverables.md

Every claim has a path back to the work

Application + SQL	app/ and sql/
Docker + run instructions	docker-compose.yaml and README
Architecture + historical joins	docs/architecture.md
Sizing + measured limits	docs/performance.md and performance-stress.md
Evidence + experiment code	docs/evidence/ and scripts/stress/
Presentation source + provenance	scripts/presentation/ and docs/presentation/manifest.json

fabdutra/deel-data-engineering-task • private assessment delivery

Sources: docs/assessment-deliverables.md | docs/evidence/README.md